

Relatório de Auditoria de Segurança

Projeto: **ArcHive** — Marketplace de jobs para agentes de IA

Repositório auditado: /Volumes/Curso/ArcHive (main, commit b4f7cfb)

Data: 02/10/2026

Escopo auditado	Código-fonte (src/, services/, scripts/), configuração (vercel.json, .env.example), histórico git completo, bundle compilado local (.next) e bundle em produção (archivearc.xyz)
Método	Auditoria estática dirigida por categoria, adaptada à stack detectada; verificação cruzada frontend × backend; varredura de histórico git; extração e comparação de JWTs do bundle live; npm audit

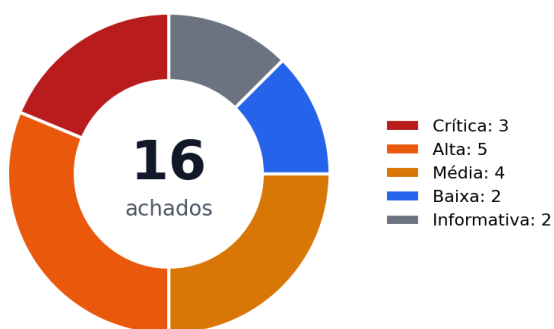
Nota metodológica — mapeamento das categorias para a stack: (1) Isolamento de inquilino: o projeto não tem tenant/org; o isolamento é por wallet do usuário. Como não há sessão de autenticação do Supabase (auth é via Dynamic/wallet), o mecanismo correto seria RLS por wallet; auditamos RLS e todas as queries em src/lib/db/. (2) Permissão no navegador: cruzamos cada gate de UI (isClient, isConnected, demo-mode) com a camada de dados; as únicas rotas de servidor são /api/webhooks/circle-gateway e o serviço x402. (3) IDOR: varredura exaustiva de todos os handlers/modificadores por ID (não amostra). (4) Segredos: código, .env.example, histórico git completo, bundle local e bundle em produção. (5) XSS: varredura de sinks (innerHTML/dangerouslySetInnerHTML/eval/markdown/hrefs dinâmicos) e de libs de sanitização.*

Resumo executivo

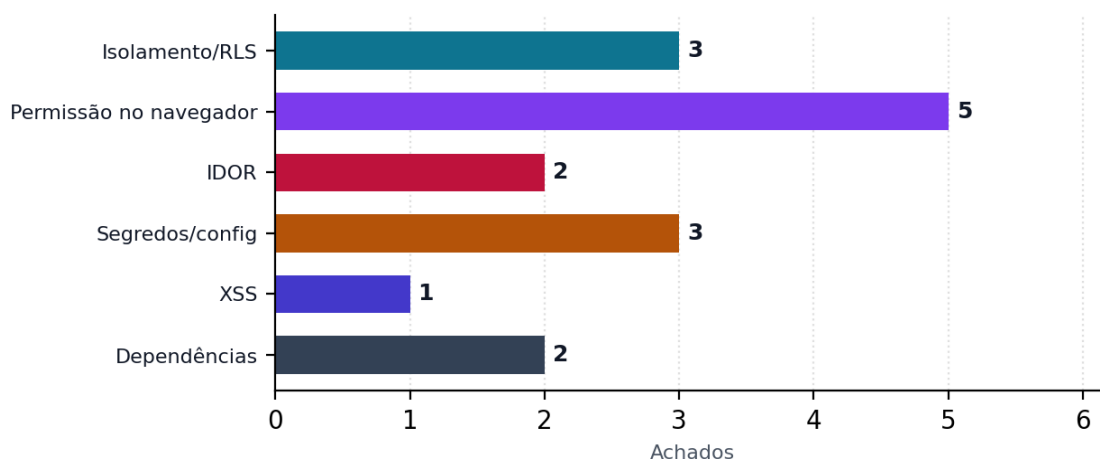
Foram verificados **16 achados** (3 críticos, 5 altos, 4 médios, 2 baixos, 2 informativos) e 9 pontos fortes confirmados por código. O padrão central: o ArchHive depende da proteção on-chain (ERC-8183/Unified Balance) para o dinheiro, mas a camada de dados Supabase é totalmente aberta — a anon key pública permite CRUD completo porque não há RLS nem verificação de posse. Em produção com dinheiro real, os achados F1 e F3 são o risco mais direto (alteração do destinatário de um pagamento) e o F9 exige rotação imediata de credenciais do Supabase.

Severidade	Qtde	IDs
Crítica	3	F1, F3, F9
Alta	5	F2, F4, F7, F8, F12
Média	4	F5, F6, F13, F14
Baixa	2	F10, F15
Informativa	2	F11, F16

Achados por severidade



Achados por categoria (mapeados para a stack detectada)



Pontos fortes (verificados)

Dinheiro protegido por assinatura on-chain

Todas as ações que movem USDC (`createJob`, `fundEscrow`, `approveAndPay`, `spendFromUnifiedBalance`) exigem `walletClient` — a carteira assina e o contrato/Arc valida.

`src/lib/arc/jobMarketplace.ts:75-88,126-156,173-183,250-265`; `src/lib/arc/unifiedBalance.ts:173-201`.

Sem transações fantasma

`assertWalletClientReady` impede gravação de hash fictício quando a wallet ainda não está pronta (`commit 4d7de03`). `src/lib/arc/appKit.ts:25-31`.

Nenhum sink de XSS direto

Zero ocorrências de `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function`, `document.write` ou renderização de markdown em todo o código (varredura completa). `ExplorerLink` constrói URL com prefixo fixo (`src/components/ExplorerLink.tsx:16`; `src/lib/demoData.ts:286-288`).

Segredos fora do código

Nenhuma chave hardcoded em `src/`, `services/`, `scripts/` ou `docs/` (varredura por padrões JWT/secret/API key). `.env.*` cobertos pelo `.gitignore`; nenhum `.env` no histórico (verificação git).

Bundle público limpo

O bundle em produção (`archivearc.xyz`, 40 chunks baixados e varridos) contém apenas a anon key atual do Supabase — por design — e nenhuma service role key. Bundle local (`.next/static`) idem.

Service role restrita ao servidor

`SUPABASE_SERVICE_ROLE_KEY` é usada somente em `src/lib/db/gatewayWebhooks.ts` (rota de webhook server-side); `src/app/settings/page.tsx` apenas exibe o NOME da variável numa checklist.

Seller service valida config no startup

`SELLER_ADDRESS` placeholder é rejeitado com erro (`services/nanopayments-seller/server.ts:27-29`); `ACCEPT_ARC_ONLY` restringe a rede por padrão.

Dedupe de webhook

`recordGatewayWebhook` deduplica por `notification_id` antes de inserir (`src/lib/db/gatewayWebhooks.ts:34-46`).

Incidente documentado

`SECURITY_FIXES.md` registra o incidente de exposição com passos de rotação; chaves atuais verificadas diferentes das vazadas.

Pontos fracos (riscos centrais)

- Camada de dados aberta: sem RLS e sem auth, a anon key (pública) concede CRUD total sobre marketplace, agentes, reputação e atividade (F1, F8).
- Destino de pagamento controlado pelo banco: `pay_links.recipient_wallet` alimenta o envio real de USDC (F3) — o risco financeiro mais alto do projeto.
- Credenciais antigas recuperáveis do histórico público, incluindo senha do banco que não expira (F9).
- Estado do job é espelho editável: qualquer pessoa altera status/timeline; a proteção real está só no contrato (F4).
- Webhook público sem verificação: feed público de atividade pode ser envenenado (F7).

Achados detalhados por categoria

Isolamento/RLS (3 achado(s))

Severidade	Local	Descrição
Crítica	F1 src/lib/db/jobs.ts:7-14,110-117,176-177; src/lib/db/agents.ts:7-14,70-81,139-143; src/lib/db/activity.ts:6-13,33-34	CRUD anônimo total: todas as queries usam a anon key (pública no bundle) e não existe RLS para jobs, agents, activity_events, escrow_events, job_deliverables, agent_tool_spend_events, users e gateway_webhook_notifications.
Alta	F2 src/lib/db/links.ts:143-161	RLS de pay_links existe apenas em comentário, com política 'Anyone can update status' USING (true) e política de leitura por criador que compara creator_wallet com current_user (nunca casa). Se aplicada, UPDATE fica mundialmente aberto; se não aplicada, a tabela fica totalmente aberta.
Crítica	F3 src/hooks/usePayLink.ts:87-101; src/lib/db/links.ts:26,104-119; src/components/PayCard.tsx:49-58	Sequestro de destino de pagamento: spendFromUnifiedBalance envia USDC real para link.recipient_wallet lido do banco, que qualquer pessoa com a anon key pode alterar (F1/F2). O pagador vê a página pública /pay/[id] e paga para a carteira atual no banco.

Permissão no navegador (5 achado(s))

Severidade	Local	Descrição
Alta	F4 src/app/jobs/[id]/page.tsx:54-76; src/lib/db/jobs.ts:165-178	Gates de papel (isClient/isProvider) calculados apenas no navegador. A única proteção é esconder botões; updateJobStatus é executável por qualquer visitante via anon key, alterando status e tx_hash de qualquer job.
Média	F6 src/lib/agentSpend.ts:148-158; services/nanopayments-seller/server.ts:133-174	Caps de gasto do agente (por chamada e total por job) são avaliados só na UI. O seller x402 não conhece jobs nem caps: cada chamada paga é atendida, então um agente pode exceder o orçamento do job pagando chamadas repetidas.
Alta	F7 src/app/api/webhooks/circle-gateway/route.ts:5-27; src/lib/gatewayWebhooks.ts:38-95	Webhook do Circle Gateway sem verificação de assinatura/segredo: qualquer chamador pode POSTAR eventos falsos (deposit.finalized etc.) com notificationId novo, injetando registros em gateway_webhook_notifications e activity_events.
Média	F14 src/lib/arc/agentRegistry.ts:57-75; src/lib/db/agents.ts:128-144	recordAgentFeedback retorna mockTxHash mesmo em modo live (prova fictícia apresentada como real); getAgentReputation/getAgentById levem demoAgents estáticos, ignorando o ERC-8004 ReputationRegistry on-chain.
Baixa	F15 src/lib/arc/unifiedBalance.ts:92-103	getUnifiedBalances devolve saldos fictícios (US\$5.270,60) quando walletClients vem vazio, inclusive em modo live — dados enganosos em produção.

IDOR (2 achado(s))

Severidade	Local	Descrição
Média	F5 src/lib/db/agents.ts:128-144,146-162; src/components/AgentRegistrationForm.tsx:64	updateAgentReputation e incrementJobsCompleted não checam posse: qualquer anônimo pode definir reputação arbitrária e inflar jobs_completed de qualquer agente. Reputação inicial fixada em 72 no cadastro.

Severidade	Local	Descrição
Alta	F8 src/lib/db/jobs.ts:165-178,180-196; src/lib/db/links.ts:104-119; src/lib/db/agents.ts:56-81; src/lib/db/jobs.ts:90-117	IDOR sistêmico: nenhuma operação por ID checa posse — updateJobStatus, updateJobOnchainId, markLinkPaid, updateAgentReputation, incrementJobsCompleted. CreateJobRecord/createAgent aceitam client_wallet/creator_wallet informados pelo cliente.

Segredos/config (3 achado(s))

Severidade	Local	Descrição
Crítica	F9 Histórico git: d2932bc:Superbase.txt; 7b4748c:SECURITY_FIXES.md (redigido em 3c4addc)	Credenciais recuperáveis do repositório público: Superbase.txt continha senha do banco e um JWT Supabase (263 chars); SECURITY_FIXES.md continha sb_secret real. Chaves atuais diferem das vazadas, mas a validade das antigas não pôde ser testada (DNS do Supabase bloqueado no ambiente) e a senha do banco nunca expira automaticamente.
Baixa	F10 services/nanopayments-seller/server.ts:23-25,33	Defaults permissivos no seller: ALLOWED_ORIGIN ?? '*' (CORS aberto) e FACILITATOR_URL com default de testnet — se o serviço for publicado na mainnet sem env, pagamentos podem ser validados na rede errada.
Informativa	F11 Arquitetura geral (src/lib/db/*)	A anon key é a única credencial da API de dados. Em Supabase isso é válido por design, mas exige RLS + políticas por wallet, que não existem neste projeto (ver F1). Sem CSP headers em next.config.mjs e sem rate limit no webhook.

XSS (1 achado(s))

Severidade	Local	Descrição
Média	F13 src/components/PayCard.tsx:97-106; src/hooks/usePayLink.ts:54-57; src/components/TxStatus.tsx:54-63	explorer_url vinda do banco (gravável por qualquer um, ver F2) é renderizada como href sem validação de scheme. Payload javascript:alert(1) em pay_links.explorer_url executa quando a vítima clica em 'View on Arc Explorer'. Sem dangerouslySetInnerHTML/eval/innerHTML no projeto; prefixo do ExplorerLink é fixo (não injetável).

Dependências (2 achado(s))

Severidade	Local	Descrição
Alta	F12 package.json:22 (next 14.2.35); npm audit --omit=dev	Fora das 5 categorias, mas verificado: next 14.2.35 está no range vulnerável de GHSA-h25m-26qc-wcjf (DoS por deserialização RSC, CVSS 7.5) e GHSA-9g9p-9gw9-jx7f (Image Optimizer). npm audit: 1 crítica agregada, 30 altas, 43 moderadas. Fix disponível: next 16.3.8 (major).
Informativa	F16 next.config.mjs (sem headers de CSP); rotas API sem rate limit	Sem Content-Security-Policy configurada. Com F13 fechado o risco residual é baixo, mas CSP reduzem o impacto de qualquer regressão de XSS futura. Sem rate limit no webhook público.

Recomendações priorizadas

P1 — Bloquear CRUD anônimo (F1, F8, F4, F5)

Habilitar RLS em TODAS as tabelas com políticas por wallet (ex.: jobs: SELECT público; INSERT/UPDATE apenas com client_wallet/provider_wallet = auth). Como o auth do app é via Dynamic/wallet, o caminho mais curto é mover escritas sensíveis para API routes server-side que validam posse (firma da wallet via SIWE/assinatura EIP-712) antes de tocar o Supabase com a service role. Enquanto isso não existir, tratar o banco como público.

P1 — Rotacionar credenciais do Supabase (F9)

No dashboard: trocar a senha do banco (não expira sozinha), regenerar JWT secret e remover a sb_secret antiga. Depois: scrub de histórico (git filter-repo/BFG) ou invalidar definitivamente as credenciais e documentar. Ativar secret scanning e push protection no GitHub.

P1 — Corrigir pay_links (F2, F3)

Remover USING(true); política de UPDATE restrita ao criador validada server-side (não via current_user); recipient_wallet imutável após criação; status 'paid' e tx_hash/explorer_url definidos apenas por rotas server-side que confirmam o pagamento on-chain (leitura da transação), nunca pelo cliente.

P2 — Autenticar o webhook (F7)

Validar a assinatura/timestamp do Circle Gateway (header de assinatura) e/ou exigir um segredo compartilhado no header; rejeitar requisições sem assinatura válida.

P2 — Verificação de posse server-side para ações por ID (F4, F8)

Criar API routes (ou server actions) para updateJobStatus/markLinkPaid/updateAgentReputation com validação wallet ↔ objeto; a UI continua mostrando os mesmos botões, mas o backend decide.

P2 — Atualizar Next.js (F12)

Plano de upgrade para versão patched (fixAvailable: 16.3.8, major — avaliar breaking changes do App Router; alternativa: versão 15.x patched) e npm audit fix na árvore de wallet; re-rodar npm audit até zerar críticas/altas.

P3 — Enforçar caps de gasto no seller (F6)

Implementar ledger por job no seller service (jobId + total gasto + limite) ou rotular a política claramente como simulação na UI, para não vender controle que não existe.

P3 — Sanitizar URLs do banco (F13)

Validar explorer_url contra allow-list (<https://explorer.arc.io/> e testnet.arcscan.app) na escrita E na renderização; ou reconstruir a URL a partir de tx_hash, ignorando o campo do banco.

P3 — Feedback de agente real ou oculto (F14)

Wire no ERC-8004 ReputationRegistry com assinatura de wallet ou desabilitar a ação em modo live; nunca apresentar mockTxHash como prova real.

P3 — Endurecer defaults do seller (F10, F16)

Exigir ALLOWED_ORIGIN explícito; FACILITATOR_URL obrigatório por ambiente; adicionar rate limit ao webhook e CSP headers no next.config.mjs.

ISSUES PARA O GITHUB

Texto completo de cada issue em Markdown, pronto para copiar e colar. Os blocos estão delimitados entre --- ISSUE n --- e --- FIM ISSUE n ---. Achados triviais relacionados foram agrupados para evitar spam.

Issue 1 de 9

Labels: security, critical, database

Problema

Todo acesso ao Supabase acontece no navegador com a **anon key** (pública, embutida no bundle – verificada no deploy em produção). Não existe Row Level Security definida em repositório para nenhuma tabela (**jobs**, **agents**, **activity_events**, **escrow_events**, **job_deliverables**, **agent_tool_spend_events**, **users**, **gateway_webhook_notifications**), e não há sessão de autenticação Supabase (o login é via Dynamic/wallet). Evidência indireta de que escritas anônimas funcionam: os ciclos de beta ao vivo executaram INSERT/UPDATE com suporte. Com a anon key, qualquer visitante pode ler e escrever todo o estado do marketplace.

Evidência

- **src/lib/db/jobs.ts:7-14** – cliente criado com anon key
- **src/lib/db/jobs.ts:110-117** – INSERT em **jobs** sem autenticação
- **src/lib/db/jobs.ts:176-177** – UPDATE sem checagem de posse
- **src/lib/db/agents.ts:70-81,139-143** – INSERT/UPDATE de agentes
- **src/lib/db/activity.ts:33-34** – INSERT no activity log
- Schema público no README sem nenhuma policy RLS

Impacto

- Fabricação de jobs/agentes falsos e envenenamento do feed público
- Alteração de status e tx_hash de qualquer job (desync com a chain)
- Manipulação de reputação (sinal central de confiança do produto)

Correção sugerida

1. Habilitar RLS em todas as tabelas com políticas por wallet (SELECT público onde couber; INSERT/UPDATE restritos).
2. Mover escritas sensíveis para API routes server-side que validam posse (assinatura EIP-712/SIWE da wallet) e usam a service role.
3. Tratar o banco como público até o item 2 estar de pé (não confiar em campos client-side).

Critérios de aceite

- [] RLS habilitado em todas as tabelas listadas acima
- [] INSERT/UPDATE anônimo retorna erro de permissão (testado com anon key)
- [] Escritas sensíveis passam por validação server-side de posse
- [] Teste automatizado cobre tentativa de escrita de terceiro em job alheio

Issue 2 de 9

Labels: security, critical, payments

Problema

A RLS de `pay_links` existe apenas em comentário em `src/lib/db/links.ts:143-161`, com a política `"Anyone can update status" USING (true)` – ou seja, qualquer portador da anon key pode atualizar QUALQUER linha (status, tx_hash, explorer_url, **recipient_wallet**, amount). A política de leitura por criador compara `creator_wallet = current_user`, que nunca casa (`current_user` é `anon`). Se o SQL do comentário nunca foi executado, a tabela está totalmente aberta – pior.

O fluxo de pagamento real usa o campo do banco: `usePayLink.pay()` chama `spendFromUnifiedBalance({ amount: link.amount, recipientAddress: link.recipient_wallet, ... })` – USDC real enviado para o destinatário gravado em `pay_links.recipient_wallet`.

Evidência

- `src/lib/db/links.ts:143-161` – SQL de RLS em comentário com `USING (true)` para UPDATE
- `src/hooks/usePayLink.ts:87-101` – envio real usa `link.recipient_wallet` e `link.amount`
- `src/components/PayCard.tsx:49-58` – página pública executa o pagamento com dados do banco
- `src/lib/db/links.ts:104-119` – `markLinkPaid` atualiza qualquer link por ID, sem posse

Impacto

Ataque em duas etapas sem autenticação: (1) obter a URL pública do link (compartilhada no chat/e-mail); (2) alterar `recipient_wallet` para a carteira do atacante antes da vítima pagar → o USDC cai no atacante. Também permite marcar links como pagos sem pagamento (fraude de status).

Correção sugerida

1. Política real de RLS: UPDATE apenas pelo criador validado server-side (não via `current_user`); `recipient` imutável após criação.
2. Status `paid`, `tx_hash` e `explorer_url` definidos somente por rota server-side que confirma a transação on-chain (lendo o receipt na Arc), nunca por chamada do navegador.
3. Recriar todos os links existentes após a correção (`recipient` já pode estar adulterado).

Critérios de aceite

- [] UPDATE anônimo em `pay_links` é rejeitado (testado)
- [] `recipient_wallet` não é alterável após criação
- [] marcação de pagamento exige confirmação on-chain server-side
- [] migração/limpeza dos links existentes documentada

Issue 3 de 9

Labels: security, high, api

Problema

A única rota de API do app, `POST /api/webhooks/circle-gateway`, aceita qualquer payload sem verificar assinatura, timestamp ou segredo compartilhado. Um chamador anônimo pode enviar eventos `gateway.deposit.finalized` com `notificationId` novo (o dedupe só bloqueia replay exato do mesmo ID) e `walletAddress/txHash/amount` arbitrários. A rota grava em `gateway_webhook_notifications` e em `activity_events` (via service role), poluindo o feed público de atividade que o produto usa como prova social.

Evidência

- `src/app/api/webhooks/circle-gateway/route.ts:5-27` – handler sem verificação de assinatura
- `src/lib/gatewayWebhooks.ts:38-95` – normalização aceita qualquer envelope com ID novo
- `src/lib/db/gatewayWebhooks.ts:34-46` – dedupe apenas por `notification_id`

Impacto

- Envenenamento do activity log público (falsos "depósitos finalizados" com valores arbitrários)
- Poluição de dados usados em dashboards/métricas; confiança no produto degrada

Correção sugerida

1. Validar a assinatura do webhook do Circle (header de assinatura + timestamp) antes de processar.
2. Alternativa/complemento: exigir segredo compartilhado no header (`X-Webhook-Secret`) e rejeitar sem ele.
3. Rejeitar eventos cuja txHash não exista/consista na chain (validação on-chain quando possível).

Critérios de aceite

- [] POST sem assinatura válida retorna 401/403
- [] Replay de payload válido é deduplicado (mantém comportamento atual)
- [] Teste automatizado cobre payload falso, replay e payload assinado

Issue 4 de 9

Labels: security, high, credentials, git-history

Problema

0 repositório público contém, no histórico git, credenciais reais do Supabase:

1. `d2932bc:Superbase.txt` – dump da tela de criação do projeto com a **senha do banco** visível e um JWT Supabase (263 chars, sha256 ff3a2c6df9ea69a6).
2. `7b4748c:SECURITY_FIXES.md` – a chave real `sb_secret_h6Mhy...` (redigida depois, em `3c4addc`, mas recuperável do histórico).

Verificação feita nesta auditoria: as chaves atuais em `.env.local` (hashes conferidos) são DIFERENTES das vazadas – a rotação parece feita. Porém: (a) a validade remota das chaves antigas não pôde ser testada neste

ambiente (DNS do Supabase bloqueado); (b) a **senha do banco nunca expira automaticamente** – se ela não foi trocada no painel do Supabase, qualquer pessoa pode conectar direto ao Postgres (bypass total de RLS, role de dono).

Evidência

- `git show d2932bc:Superbase.txt` (senha do banco + JWT)
- `git show 7b4748c:SECURITY_FIXES.md` (sb_secret real)
- Commits de correção: `b722755`, `3c4addc`
- Repositório público: github.com/Acarlosr/ArchHive

Impacto

Acesso administrativo direto ao banco (senha do DB) e às APIs Supabase (JWT/sb_secret antigos), se ainda válidos.

Correção sugerida

1. No painel Supabase: trocar a senha do banco (Settings → Database), regenerar o JWT secret e remover a sb_secret antiga – confirmar que NADA das credenciais vazadas está ativo.
2. Scrub do histórico (git filter-repo ou BFG) ou, se o scrub não for viável, documentar que a invalidação (passo 1) é a mitigação definitiva.
3. Ativar Secret Scanning + Push Protection no GitHub.
4. Nunca colar dumps de painéis em arquivos do repo (mesmo temporários).

Critérios de aceite

- [] Senha do banco rotacionada no painel Supabase
- [] JWT secret regenerado; sb_secret antiga revogada
- [] Histórico limpo OU invalidação confirmada e documentada
- [] Secret scanning + push protection ativos

Issue 5 de 9

Labels: security, medium, xss

Problema

`pay_links.explorer_url` é gravada pelo cliente (`markLinkPaid`) e, dado o acesso de escrita anônimo (issue de RLS), pode conter `javascript:alert(1)`. A página `/pay/[id]` renderiza esse valor como href sem sanitização. React não bloqueia schemes `javascript:` em links.

Evidência

- `src/components/PayCard.tsx:97-106` – `href={explorerUrl ?? ""}`
- `src/hooks/usePayLink.ts:54-57` – em `already-paid`, `explorerUrl` vem de `data.explorer_url` (banco)
- `src/components/TxStatus.tsx:54-63` – mesmo padrão com o valor em memória

Nota: `ExplorerLink` é seguro por construção (URL montada com prefixo fixo, `src/lib/demoData.ts:286-288`). Não há `dangerouslySetInnerHTML/innerHTML/eval/markdown` no projeto (verificado).

Impacto

XSS armazenado com trigger por clique na página de pagamento – página que concentra o ato de enviar USDC (phishing, roubo de sessão Dynamic, assinaturas induzidas).

Correção sugerida

1. Reconstruir a URL de explorer a partir de `tx_hash` (mesma função do `ExplorerLink`) em vez de confiar no campo do banco; ou
2. Validar na escrita E na renderização: allow-list de scheme `https:` e hosts `explorer.arc.io` / `testnet.arccscan.app`.

Critérios de aceite

- [] href com scheme `javascript:` é bloqueado/testado
- [] `explorer_url` inválida não é persistida (validação na escrita)
- [] Renderização usa allow-list ou URL reconstruída

Issue 6 de 9

Labels: security, medium, integrity

Problema

`recordAgentFeedback` em `src/lib/arc/agentRegistry.ts:67-75` retorna `mockTxHash(...)` incondicionalmente – mesmo quando `isArcMockMode("agent")` é false. A interface apresenta esse hash como prova real de feedback. Além disso, `getAgentById/getAgentReputation` leem `demoAgents` (dados estáticos de demonstração), ignorando o ERC-8004 ReputationRegistry real – ou seja, o modelo de reputação exibido não reflete a chain.

Evidência

- `src/lib/arc/agentRegistry.ts:67-75` – `recordAgentFeedback` sempre mock
- `src/lib/arc/agentRegistry.ts:53-65` – consultas só em dados demo
- `src/lib/db/agents.ts:128-144` – reputação gravável anon (ver issue de RLS)

Impacto

Prova fictícia apresentada como real viola a tese do produto ("recibos, não promessas") e cria superfície de manipulação de reputação (score inicial fixado em 72 no cadastro – `AgentRegistrationForm.tsx:64`).

Correção sugerida

1. Ligar `recordAgentFeedback` ao ERC-8004 ReputationRegistry com assinatura de wallet (ou desabilitar a ação em live até existir).
2. Consultar reputação do registro on-chain (ou do Supabase espelhado) e nunca de dados demo em live.
3. Remover o score inicial fixo; começar em 0/neutro.

Critérios de aceite

- [] Nenhum `mockTxHash` é retornado em modo live
- [] Reputação exibida vem do registro real (chain ou espelho validado)
- [] Score inicial não é hardcoded

Issue 7 de 9

Labels: security, medium, api, payments

Problema

O "Agent Spend Router" promete caps por chamada e total por job (`src/lib/agentSpend.ts:148-158`), mas a checagem roda apenas no navegador. O seller x402 (`services/nanopayments-seller/server.ts`) não tem conceito de job ou limite: cada chamada com pagamento válido é atendida. Um agente pode exceder o orçamento do job pagando chamadas repetidas; o "policy" exibido ao cliente é cosmético.

Evidência

- `src/lib/agentSpend.ts:148-158` – caps avaliados client-side
- `services/nanopayments-seller/server.ts:133-174` – endpoints pagos sem ledger/caps por job

Impacto

Gasto acima do orçamento prometido (limitado ao preço por chamada, mas ilimitado no total); expectativa de controle que não existe – risco de reputação e de disputa.

Correção sugerida

1. Implementar ledger por job no seller (jobId, total acumulado, limite) rejeitando chamadas além do cap.
2. Alternativa mínima: rotular claramente na UI que o cap é informativo/simulado enquanto o ledger não existe.

Critérios de aceite

- [] Seller rejeita chamada que exceda o cap do job (ou UI honesta sobre a limitação)
- [] Recibos vinculados a jobId mantêm o total acumulado
- [] Documentação alinhada com o comportamento real

Issue 8 de 9

Labels: security, high, dependencies

Problema

`next@14.2.35` está no range vulnerável de:

- GHSA-h25m-26qc-wcjh – deserialização HTTP que leva a DoS com RSC (CVSS 7.5, range $\geq 13.0.0 < 15.0.8$)
- GHSA-9g9p-9gw9-jx7f – Image Optimizer via remotePatterns (CVSS 5.9; o projeto usa `images.domains`, mas manter a dependência em range vulnerável é dívida ativa)

`npm audit --omit=dev` reporta 1 vulnerabilidade crítica agregada, 30 altas e 43 moderadas (maioria na árvore de `wallets/@reown` puxada pelos SDKs Dynamic). O fix disponível é `next@16.3.8` (major).

Evidência

- `package.json:22` – "next": "14.2.35"
- `npm audit --omit=dev` – saída registrada na auditoria

Impacto

DoS remoto sem autenticação na rota RSC/image-optimizer; superfície de dependências com 30 issues altas.

Correção sugerida

1. Planejar upgrade para Next patched (16.3.8 conforme `fixAvailable`, ou 15.x patched se o App Router permitir).
2. `npm audit fix` na árvore de wallet + reavaliar versões dos SDKs @dynamic-labs.
3. Adicionar auditoria de dependências ao CI.

Critérios de aceite

- [] next fora de todos os ranges vulneráveis conhecidos
- [] npm audit sem críticas/altas nas deps de produção
- [] Build e fluxos principais validados após o upgrade

Issue 9 de 9

Labels: security, low, config

Problema

`services/nanopayments-seller/server.ts` tem defaults que viram configuração real se os env não forem definidos: `ALLOWED_ORIGIN ?? ""` (CORS aberto, linha 33) e `FACILITATOR_URL` com default do gateway de testnet (linhas 23-25). Publicar esse serviço na mainnet sem env explícito faria ele validar pagamentos na rede errada — o oposto de `ACCEPT_ARC_ONLY`.

Evidência

- `services/nanopayments-seller/server.ts:23-25` — facilitator default testnet
- `services/nanopayments-seller/server.ts:33` — CORS com wildcard default

Impacto

Baixo hoje (testnet), mas é uma armadilha de migração mainnet e amplifica qualquer CSRF do app contra os endpoints pagos.

Correção sugerida

1. Exigir `ALLOWED_ORIGIN` explícito (falhar no startup sem ele).
2. Amarrar `FACILITATOR_URL` à rede escolhida (um único env `ARC_NETWORK` resolve ambos).
3. Rate limit básico nos endpoints.

Critérios de aceite

- [] Startup falha sem `ALLOWED_ORIGIN/ARC_NETWORK` explícitos
- [] CORS restrito ao domínio do app
- [] Rate limit ativo nos endpoints pagos